14.3 Importar em gc-dev e conciliar — status de entrega

Bloco 14.3 Importar em gc-dev e conciliar · Sprint 14 — Migração da planilha e go-live · Data: 2026-10-26 · Branch: `main` · Ambiente: gc-dev (`gzbmhgnpoehormnidmgg`)

Resumo

Preparado, aguarda a planilha real. A "Gestão Contratos.xlsx" não chegou, então nada foi importado e nada foi conciliado de verdade. Tudo roda sobre a planilha falsa até ela chegar. As decisões do 14.1 foram tomadas por análise (delegação do Breno em 26/10), e a aplicação delas no código foi feita em 26/10. gc-prod fora do escopo por decisão do Breno (26/10): o bloco fecha em gc-dev; o restante é registro para quem for operar prod. O que dá para fazer sem ela está pronto:

- a comparação, pura e testada (`src/lib/conciliacao.ts` , 8 testes);
- a linha de comando que lê a planilha, lê o gc-dev e diz o que não bate (`scripts/importacao/conciliar.mjs`);
- um ensaio só de leitura sobre a planilha falsa do 14.2.

No dia em que a planilha chegar, o bloco é: ajustar o mapa, importar, conciliar, corrigir, repetir até bater, e conferir 3 obras na tela.

1. As atividades do card

ATIVIDADE	ESTADO
Rodar a importação completa em gc-dev	⏳ aguarda a planilha. A ferramenta é a do 14.2 (`importar-planilha.mjs --gravar`)
Conciliar contagens (8 obras, 24 propostas, 120 itens, 9 NFs, 248 FDs)	🔧 `conciliar.mjs` compara planilha × banco × referência. Os números do card são a referência padrão
Conciliar valores (R$ 627.959,83 de NF emitida, R$ 489.498,13 de recebido, R$ 2.060.290,51 de FD a descontar)	🔧 idem, no total e por obra, com tolerância de um centavo
A view `receitas_obra` bate com o Resumo da planilha	🔧 por obra. O FD vem de `obras_financeiro` , porque `receitas_obra` não tem coluna de FD (seção 3)
Validar visualmente 3 obras	⏳ manual. O script sugere as 3 de maior NF emitida, com a rota

ATIVIDADE	ESTADO
Repetir até bater	🔧 `--desfazer` (14.2) + importar de novo + conciliar. Saída 1 enquanto não bater

Legenda: ⏳ não feito, depende da planilha · 🔧 ferramenta pronta, não exercitada com dado real.

2. Como usar

```
# 1. simular e ler as ignoradas (14.2)
node --env-file=.env.local scripts/importacao/importar-planilha.mjs "Gestão Contratos.xlsx"

# 2. gravar em gc-dev (14.2)
node --env-file=.env.local scripts/importacao/importar-planilha.mjs "Gestão Contratos.xlsx" --gravar

# 3. conciliar; saída 0 só se tudo bater
node --env-file=.env.local scripts/importacao/conciliar.mjs "Gestão Contratos.xlsx"

# não bateu: desfazer, ajustar mapa ou regra, voltar ao 2
node --env-file=.env.local scripts/importacao/importar-planilha.mjs "Gestão Contratos.xlsx" --desfazer
```

Opções do `conciliar.mjs`:

OPÇÃO	O QUÊ
`--referencia ref.json`	os números de referência. Vence a chave `referencia` do mapa, que vence a do card
`--sem-referencia`	compara só planilha × banco
`--ate AAAA-MM-DD`	data de corte. Tem de ser a mesma da importação
`--mapa`, `--autor`	como na importação. O autor define a empresa consultada
`--relatorio saida.json`	relatório completo. Padrão: `conciliacao-<data>.json` no diretório temporário

O `ref.json`:

```
{ "contagens": { "obras": 8, "propostas": 24, "itens": 120, "notas": 9, "fds": 248 },
  "valores": { "nf_emitida": 627959.83, "recebido": 489498.13, "fd_a_descontar": 2060290.51 },
  "por_obra": { "<código>": { "nf_emitida": 0, "recebido": 0, "fd_a_descontar": 0 } } }
```

O `por_obra` é opcional. O card só dá o total.

3. O que o `conciliar.mjs` confere

O script **só lê** o banco. Cada linha tem até três fontes: a planilha, o banco e a referência. A linha está ok quando todas as fontes presentes batem: contagem exata, valor com tolerância de um centavo.

GRUPO	PLANILHA	BANCO	REFERÊNCIA
Contagens	registros montados	linhas achadas pela chave de negócio do 14.2	obras, propostas, itens, NFs, FDs
Totais	`totaisDosRegistros`	soma das tabelas, nas obras da planilha	os três valores do card
Por obra	NF, recebido e FD por obra	idem, por obra	`por_obra`, se houver
Propostas	valor, soma de quantidade × unitário	`valor_total`, soma de `itens.valor_total`	—
Status das NFs	`statusNfDaPlanilha`	o status que o trigger dos pagamentos calculou	—
Views × Resumo	aba Resumo, por obra	`receitas_obra` e `obras_financeiro`	—

Detalhes das regras:

- **"Faltando no banco"**: a entidade não tem nenhuma linha no banco, ou a obra não foi achada. Obra achada sem NF vale R$ 0,00, e não "faltando".
- **Recebido e FD no banco** são somados por obra, não pela marca `[importacao:…]`. É o que a tela e as views mostram. Pagamento ou FD lançado à mão numa obra importada aparece como diferença, de propósito.
- **Status de NF**: tem de ser igual. A exceção é planilha "emitida" (Atrasada, Em aberto) com o banco em `paga_parcialmente`: a planilha não tem como dizer "parcial".
- **Propostas.** Duas linhas são só aviso, sem reprovar: valor da planilha × valor do banco, e soma dos itens. O trigger do 5.6 faz o banco valer a soma (14.2, seção 6). A linha que reprova é outra: no banco, o valor da proposta tem de ser a soma dos itens. Proposta sem item no banco não entra nessa linha.
- **Views × Resumo.** NF emitida e recebido saem de `receitas_obra` (`total_nfs_emitidas`, `total_recebido`). O saldo sai de `saldo_pendente`, só se o Resumo tiver coluna de saldo. O FD sai de `obras_financeiro.total_fd_aceito`: a `receitas_obra` (migration inicial, sem redefinição depois) não tem coluna de FD.
- **Sem aba Resumo legível**, a view é comparada com os registros montados, e a saída diz isso. O mapa da aba Resumo é **provisório**, como o resto. Fica no script e pode ser trocado pela chave `resumo` do mapa: abas "Resumo", "Resumo Geral" e "Resumo por Obra"; linha "Total..." ignorada.

- **Saída 1** quando houver qualquer um destes:
  - linha divergente que não seja aviso;
  - status de NF divergente;
  - aba com problema de leitura.
- **Conferência visual.** No fim, o script lista as 3 obras de maior NF emitida no banco, com a rota `/obras/<id>`.

4. Arquivos

ARQUIVO	O QUÊ
`src/lib/conciliacao.ts`	regra pura: `conciliarLinha`, `conciliar`, `entradasPorObra`, `conferirStatusNf`, `resumoDaConciliacao`, `formatarValor` (R$ pelo `formatCurrency` de `format.ts`)
`src/lib/conciliacao.test.ts`	8 testes: tolerância, as três fontes, faltando de cada lado, por obra, status de NF, aviso × divergência, formatação
`scripts/importacao/conciliar.mjs`	a linha de comando: mesmo `lerPlanilha` + `montarRegistros` da importação, gc-dev pela chave de serviço com `gc-dev-guard`

Nenhum arquivo existente foi alterado.

5. O ensaio, só de leitura

Gerei a planilha falsa do 14.2 (`planilha-falsa.mjs`) no diretório temporário e rodei o `conciliar.mjs` contra o gc-dev **sem importar nada**. Havia uma rodada de validação em curso usando o banco. O esperado: tudo "faltando no banco" e saída 1.

RODADA	RESULTADO	SAÍDA
referência do card (padrão)	0/34 linhas ok · 28 divergentes · 6 avisos · status de NF 0/2 · além do "faltando no banco", "planilha difere da referência" em todas as 6 linhas com referência (3 obras contra 8, R$ 6.250,00 contra R$ 627.959,83…)	1
`--referencia` com os números da planilha falsa	0/34 linhas ok · 28 divergentes · 6 avisos · status de NF 0/2 · motivo único nas 34 linhas: "faltando no banco"; nas 2 NFs: "NF faltando no banco"	1
`--sem-referencia`	0/34 linhas ok · 28 divergentes · 6 avisos · status de NF 0/2	1

O ensaio também mostrou o aviso da aba Resumo: a da planilha falsa só tem uma frase, então a leitura respondeu "não achei a coluna de obra", e a view foi comparada com os registros.

Outras checagens, todas nos arquivos novos:

CHECAGEM	RESULTADO
`node --test … src/lib/conciliacao.test.ts`	8/8
`npm test` (a suíte inteira, com os 8 novos)	420/420
`tsc --noEmit`	sem erro
`eslint` nos três arquivos	sem erro

O caminho feliz não foi provado: nada foi gravado, então nenhuma linha "ok" com o banco cheio foi vista. Isso fica para a integração no plano (seção 6).

6. Validação

Validado em 2026-10-27 (o que a planilha falsa alcança). A rodada completa passou nas sete camadas.

Rodada final de 2026-10-27 (`caffeinate -ims npm run validar`)**: passou nas sete camadas.**

CAMADA	RESULTADO	DESTE BLOCO
estático	limpo (`tsc --noEmit` + `next lint`)	limpo nos arquivos novos
unitário	**432 casos**	**8 testes novos** de `src/lib/conciliacao.test.ts`
build	**66 rotas**, iguais à baseline	o bloco não tem rota
runtime	**327/327 rotas** + **348 células** da varredura de URL direta	o bloco não tem rota
dados	**65/65 checagens** (1 sem registro em gc-dev); matriz de RLS: **456 células** de tabela e **108** de bucket	o bloco não tem query de tela
escrita	**529/529 passos**, com o ensaio da importação **17/17** dentro dela	conciliação dentro do ensaio: **41/41 linhas**; o ensaio tem **17 passos**
navegador	**281/281 passos**; screenshots em `/tmp/gc-validacao/shots`	o bloco não tem diálogo

A importação e a conciliação **com a planilha real** não rodaram e seguem pendentes (seção 7).

7. O que falta, e de quem

- **A planilha real (equipe).** Sem ela não há importação nem conciliação de verdade.
- **Decisões do 14.1: aplicadas em 26/10** (importador 17/17, conciliação 41/41), as mesmas do 14.2:
  1. a tabela de equivalências de cliente;
  2. nada vai para contratos;

3. a regra da cascata;
4. links do Drive;
5. a data de corte.

A data de corte mexe direto nos totais da conciliação.

- **Confirmar os números do card (equipe).** Não se sabe:
  - de que data são;
  - se "NF emitida" exclui as canceladas (o script exclui, como as views);
  - se "FD a descontar" é o `valor_descontar` (o script usa este) ou o valor bruto;
  - se os 248 FDs são linhas da aba ou lançamentos.

  Até a confirmação, os números do card são referência **a confirmar**.
- **O layout da aba Resumo (planilha).** O mapa dela é palpite: nome da aba e cabeçalhos.
- **Conferência visual de 3 obras (manual).** Não foi feita. Depois da importação, abrir as 3 obras que o script sugere, aba Financeiro, e comparar com o Resumo.
- **Integração no plano de validação: feita (não é mais pendência).** A conciliação está dentro de `validar-importacao.mjs`: conciliação 41/41 no ensaio; o ensaio passou de 9 para 17 passos. O trecho abaixo é o desenho original, mantido como registro. Desenho: acrescentar a conciliação ao `scripts/validacao/validar-importacao.mjs`, dentro do `try`, logo depois do passo "rodar de novo não duplica" e antes do `catch`:

```
// 14.3: a conciliação bate com o que acabou de ser gravado
const refFalsa = join(tmpdir(), `referencia-falsa-${Date.now()}.json`)
writeFileSync(refFalsa, JSON.stringify({
  contagens: { obras: 3, propostas: 3, itens: 3, notas: 2, fds: 2 },
  valores: { nf_emitida: 6250, recebido: 4000, fd_a_descontar: 4500 },
}))
const mapaComEq = join(tmpdir(), `mapa-ensaio-${Date.now()}.json`)
writeFileSync(mapaComEq, JSON.stringify(mapa))
let conc = { status: 0, stdout: '' }
try {
  conc.stdout = execFileSync(process.execPath,
    [new URL('../importacao/conciliar.mjs', import.meta.url).pathname, arquivo,
     '--mapa', mapaComEq, '--referencia', refFalsa], { env: process.env, encoding: 'utf8'
})
} catch (e) { conc = { status: e.status, stdout: String(e.stdout) } }
checar('14.3: a conciliação bate (saída 0, BATEU)', conc.status === 0 &&
conc.stdout.includes('BATEU'),
  conc.stdout.split('\n').filter((l) => l.includes('DIVERGE')).join(' | '))
```

Falta também acrescentar `writeFileSync` ao import de `node:fs`, ao lado do `readFileSync`. O ensaio passa de 9 para 10 passos (depois, com as decisões do 14.1, 17).

Antes de integrar, rodar uma vez à mão contra a planilha falsa gravada. **Estava por provar**: pelo desenho, as linhas de proposta que podem divergir na planilha falsa são avisos e não reprovam. P2 perde o item 2 (unidade "litro", ignorado), e P3 não tem item.

- **Índice de `docs/README.md`.** Não foi atualizado: o arquivo estava com alteração de outra frente no working tree.
- **Produção.** Registro para quem for operar prod: a conciliação, como a importação, só roda em gc-dev (`gc-dev-guard`). A conciliação do go-live (14.4) é ensaiada em gc-dev: gc-prod fora do escopo por decisão do Breno (26/10): o bloco fecha em gc-dev; o restante é registro para quem for operar prod.